

Средства симуляции ЦП и ОС

Лекция №17



Державин Андрей
Шурыгин Антон

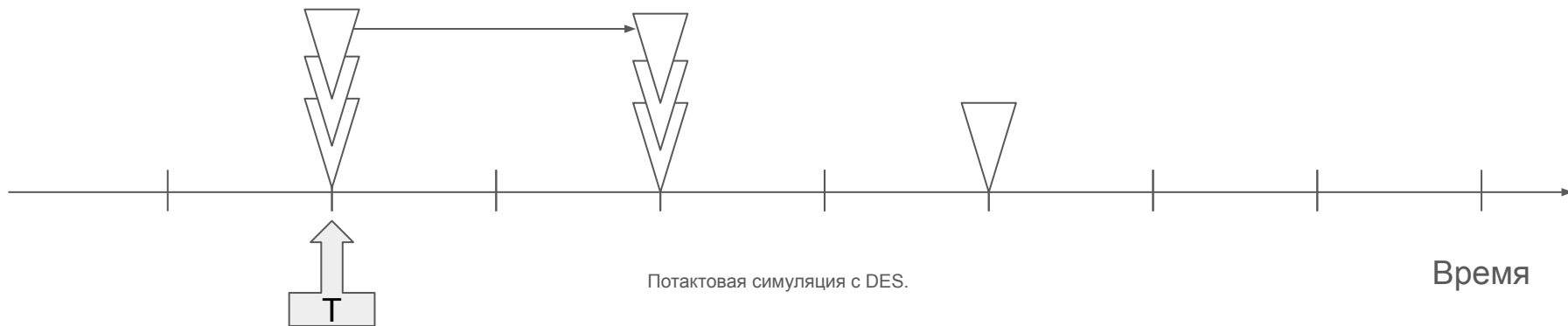
Потактовые модели

Потактовые модели

- Рассмотрим сценарии моделирования
 - Одно устройство, события на каждом такте
 - Интерпретатор+ (CPU)
 - Много устройств, события значительно реже
 - Система дискретных событий (DES)
 - Много устройств, события на каждом такте
 - Сценарий хорошо ложится на модели цифровых схем
 - Каждый внутренний блок есть устройство

Потактовые модели

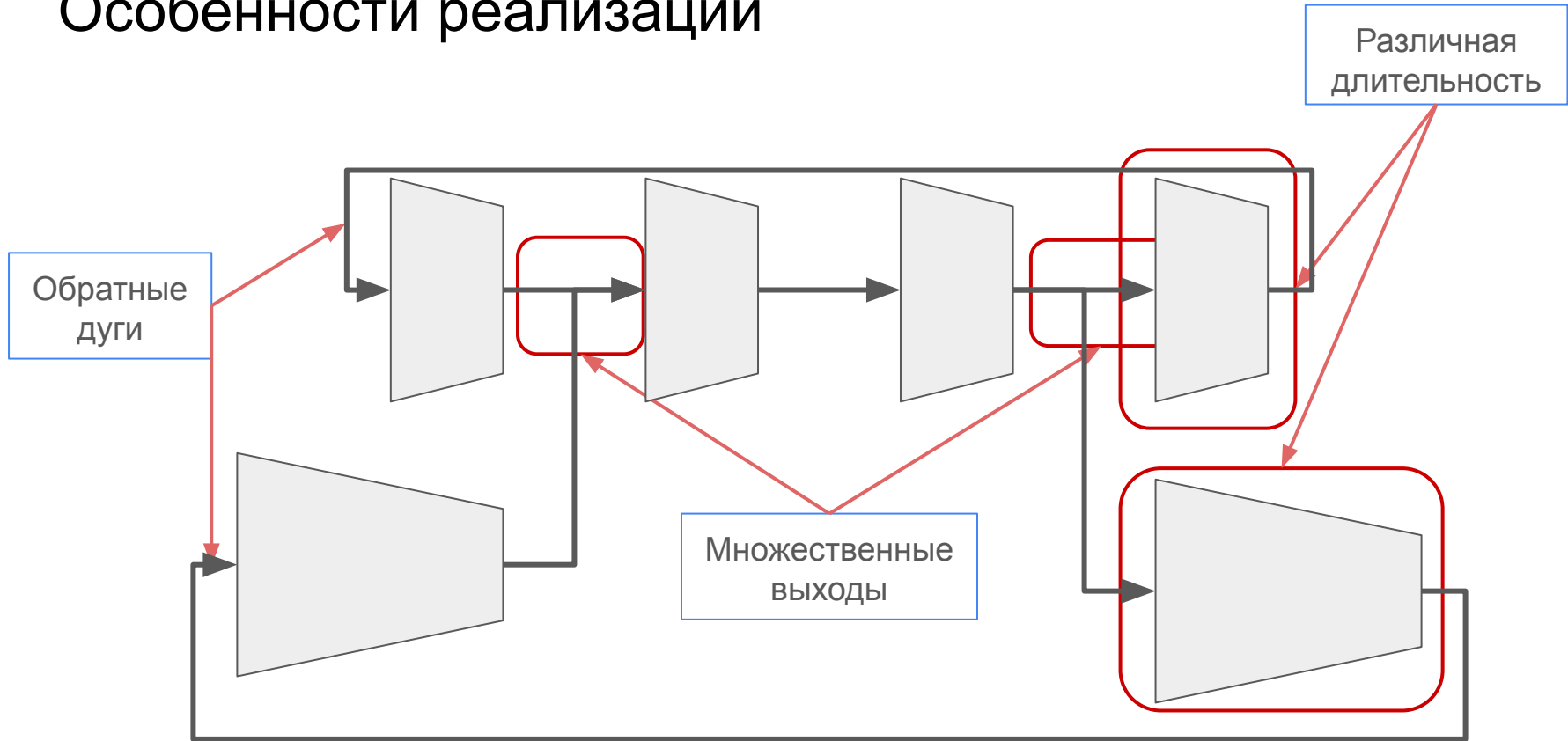
- Используем систему очередей, как для таймера
 - Каждый такт происходит много событий
 - Тяжело отслеживать перемещение данных и взаимодействие устройств
- Современные цифровые схемы - синхронные
 - Темп задаётся единым тактовым генератором
 - Состояния всех элементов меняются по фронту(спаду) тактового сигнала
 - Результаты текущего такта не будут переданы до наступления следующего



Особенности реализации

- Узлы реальной системы работают параллельно
 - Симулятор обрабатывает узлы последовательно
- Связи между узлами могут быть сложными
 - Выход может быть связан с предыдущим узлом, или даже с самим собой
 - Сложно установить глобальный порядок симуляции
- Некоторые операции занимают более 1 такта
- Узлы могут состоять из более мелких устройств
- Необходим подход, учитывающий все особенности

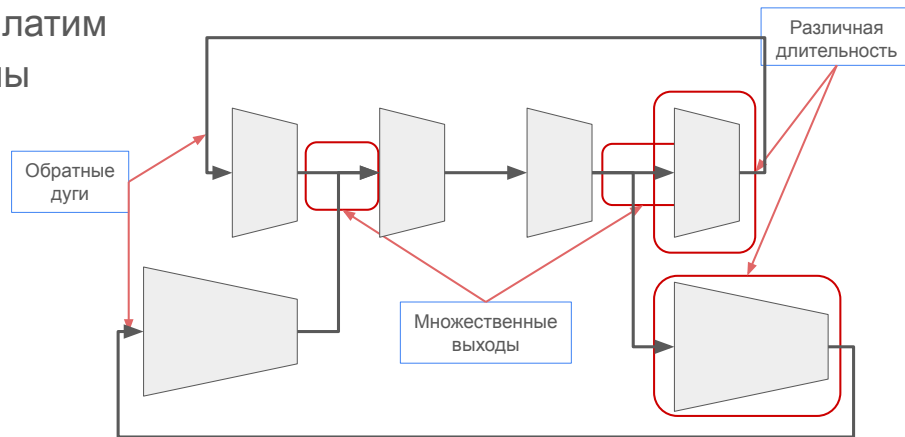
Особенности реализации



Пример соединения узлов в случайной цифровой синхронной схеме

Особенности реализации

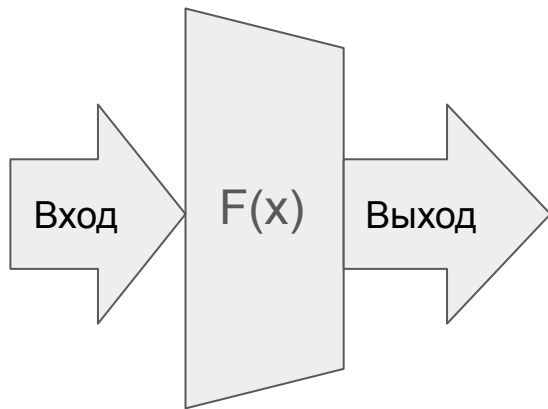
- Зависимости между узлами при последовательной модели:
 - На N такте узлы обрабатывают данные, полученные в результате операции на N-1 такте
 - Часть узлов могла закончить выполнять N-ый такт, а часть – находится на N-1 такте
- Проблема изоляции данных между тактами
 - Глобальный порядок симуляции?
 - Чем больше система => тем больше платим
 - Циклические зависимости недопустимы



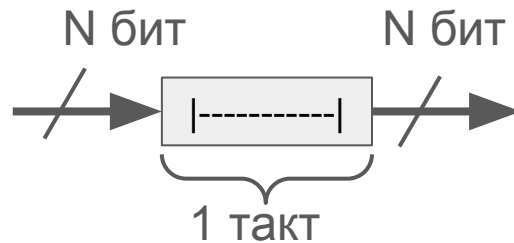
Общая схема симуляции

Схема симуляции

- Попробуем разделить функциональную и временнУю части
- Узлы выдают результат мгновенно (ведут себя как функции)
- Задержка моделируется с помощью линии задержки (*порты*) с фиксированной длиной и пропускной способностью



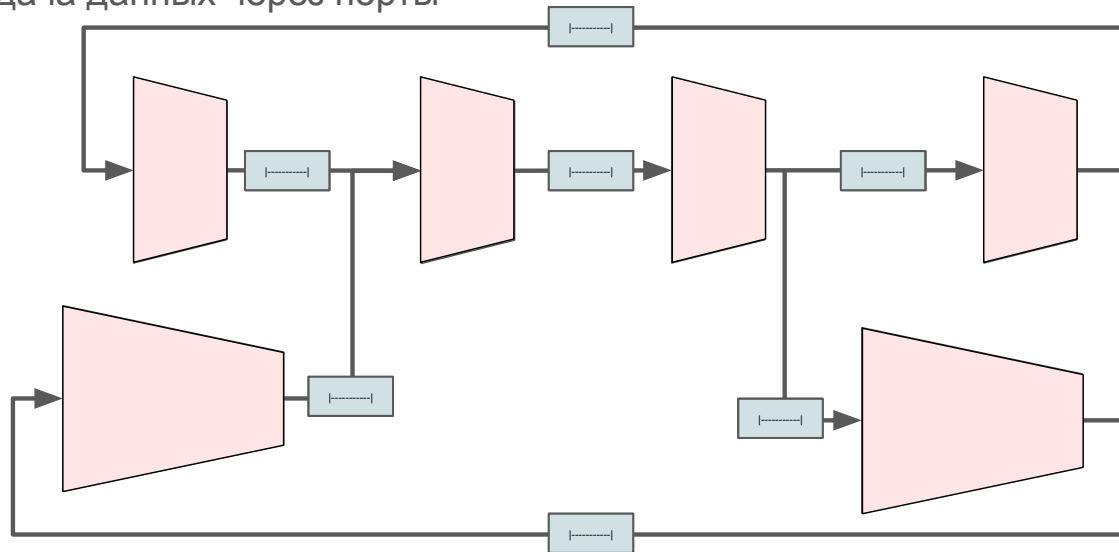
Функциональный блок



Порт

Схема симуляции

- Нельзя напрямую соединять два функциональных блока
- Симуляция происходит в два этапа:
 - Вычисление функциональных блоков
 - Передача данных через порты



Поэтапная схема работы симуляции с использованием портов для симулируемого такта

Реализация потактовых моделей

Реализация моделей. Порт

- Как мы можем реализовать модель порта?
- Что-то похожее мы уже делали...

```
template <typename T>
class Port {
    T m_input{}, m_output{};
public:
    void input(const T &data) { m_input = data; }
    const T &output() const { return m_output; }
    void on_clk() { m_output = m_input; }
};
```

Готовность данных

- Длительность операций может превышать один такт
- Каждый порт читает значение на входе, не проверяя, корректно ли значение
- Для унификации и безопасности принято добавлять бит валидности

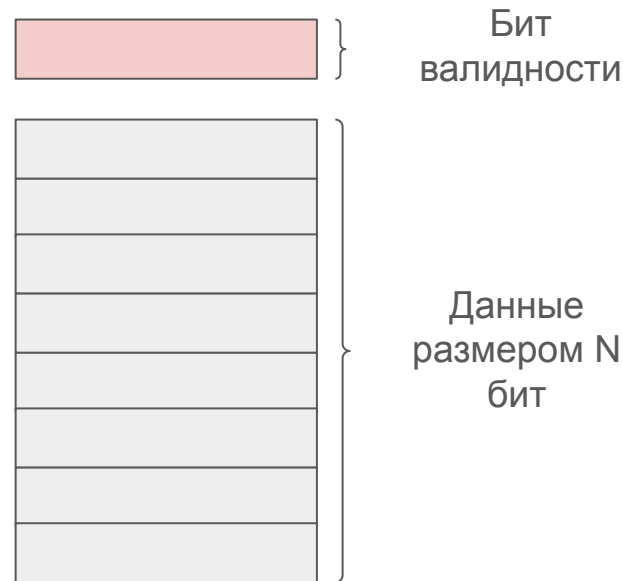


Рис. 5. Передаваемые данные

Реализация моделей. Порт

- Операции могут занимать более одного такта
- Как должен вести себя порт?
- Добавляем признак валидности данных

```
template <typename T>
class Port {
    std::optional<T> m_input{};
    std::optional<T> m_output{};
public:
    void input(const T &data) { m_input = data; }
    const auto &output() const { return m_output; }
    void on_clk() { m_output = m_input; }
};
```

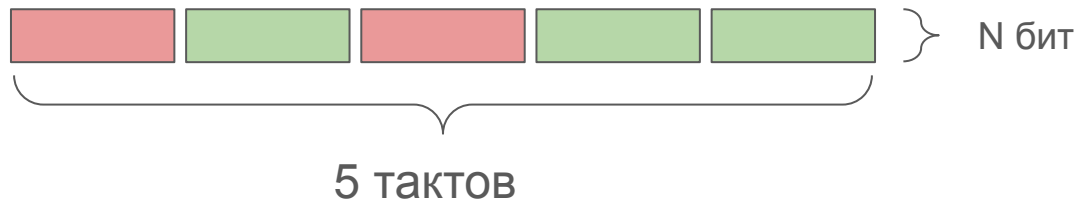
Реализация моделей. Длительные задержки.

- Как моделировать более длительные задержки?
- Соединим несколько последовательных портов



Реализация моделей. Длительные задержки.

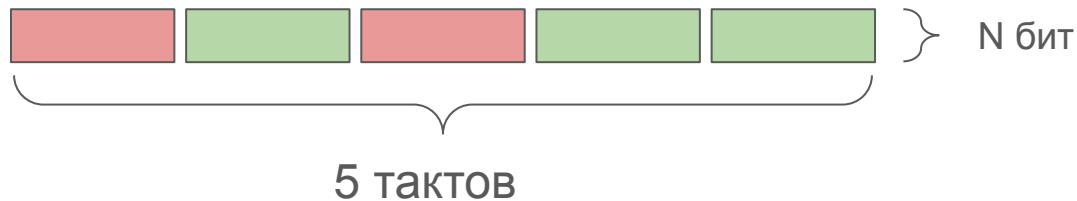
- Как моделировать более длительные задержки?
- Соединим несколько последовательных портов
- Пропускная способность портов



Последовательное соединение одноканальных портов. Зелеными блоками обозначены валидные данные.

Реализация моделей. Длительные задержки.

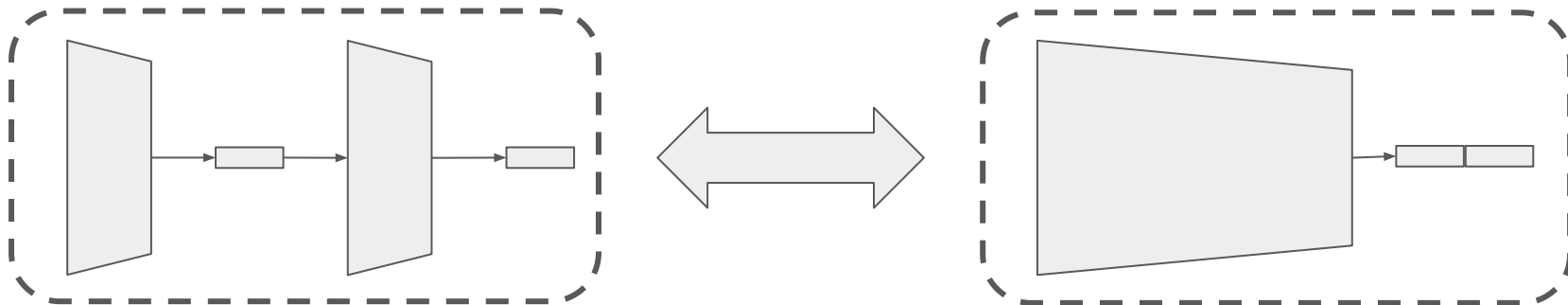
- Все ли хорошо?
 - Можем нарушить изолированность стадий
- Как решить эту проблему?
 - Разделить порты повторителями
 - Установить порядок симуляции портов
 - Объединить в один многотактный порт



Последовательное соединение одноктактных портов. Зелеными блоками обозначены валидные данные.

Композиция узлов

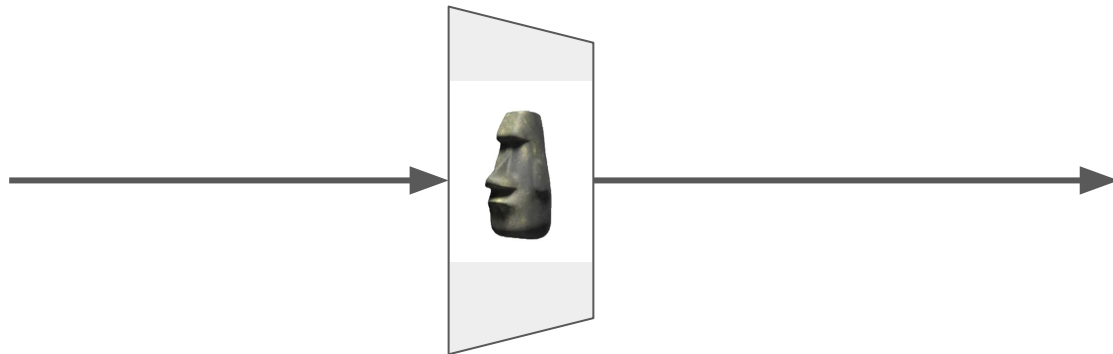
- Как можно оптимизировать симуляцию, если внутренние процессы между двумя узлами не представляют интереса?
 - Можно объединить их в композицию



Использование композиции устройств в потактовой модели,
построенной с помощью портов.

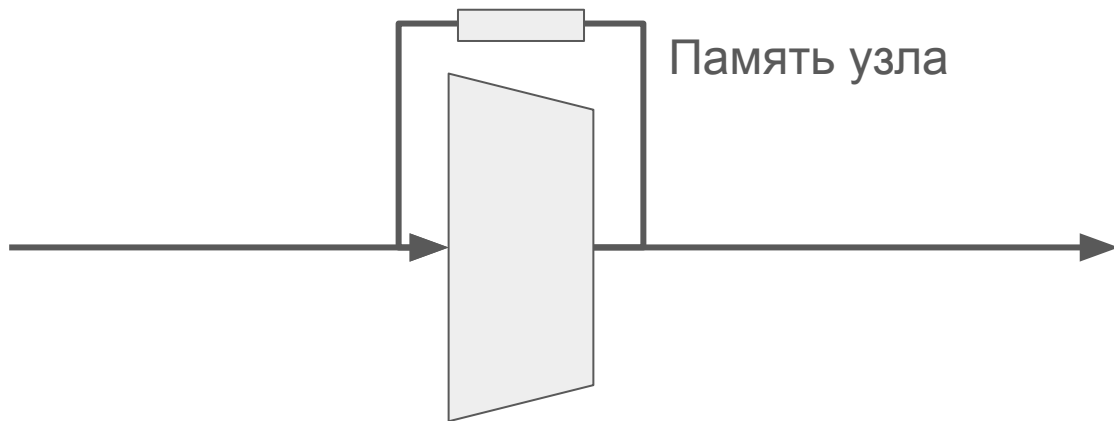
Реализация моделей.

- Уделим внимание функциональным блокам



Состояние функциональных узлов

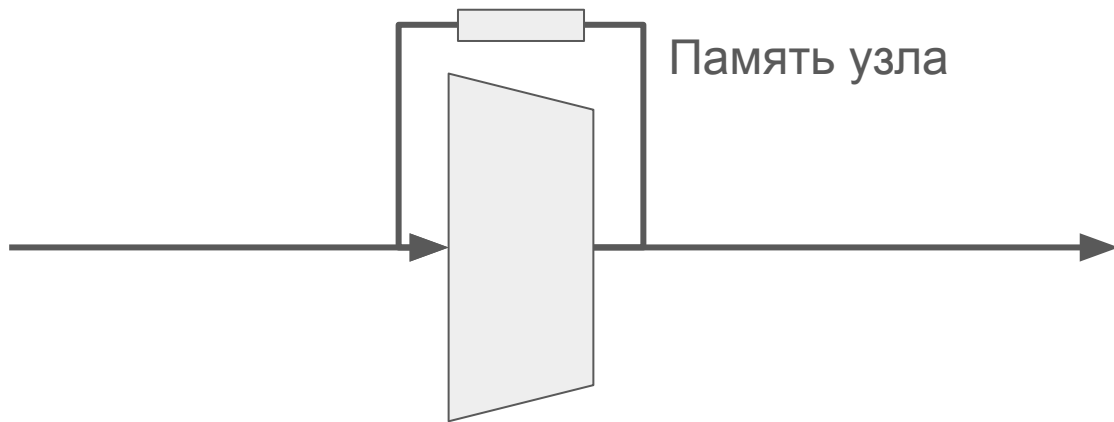
- Для реализации некоторых моделей устройств требуется хранить внутреннее состояние узлов
- Используем порты с обратной связью!



Хранение состояния узла с помощью задержки на один такт.

Память узлов

- Если состояние узла очень объемное, то перегон памяти будет медленным
- Отмечаем изменившиеся данные на выходе



Хранение состояния узла с помощью задержки на один такт.

Реализация моделей. Ускорение симуляции

- Имеющаяся реализация сильно замедляет процесс симуляции
 - Функциональная симуляция ~100 MIPS
 - Потактовая симуляция ~1 MIPS
- Все узлы работают параллельно и независимо (благодаря двум этапам)
- Как можно ускорить симуляцию?
 - Параллельная симуляция
 - Число узлов может превышать число ядер процессора
 - Использование FPGA (ПЛИС)
 - Популярное решение
 - Сложно переносить модель

Реализация моделей. Модели задержек

- Создание потактового симулятора обычно начинается с реализации функциональной модели
- Хочется переиспользовать имеющуюся функциональность для уменьшения объема работы
- Моделируем только время конкретных операций
- Функциональная часть остается за функциональной моделью

Введение в get5

История



- Был симулятор M5
 - Разработка Мичиганского университета, представлен впервые на IEEE MICRO, 2006
 - DES (англ. discrete event-driven system) симуляторный инструментарий для сбора статистики
 - Набор предопределенных моделей: CPU, кэши, девайсы
 - Использовался для моделирования сети (TCP/IP)
- Был симулятор GEMS
 - Разработка Висконсинского университета, представлен впервые в SIGARCH, 2005
 - Функциональная база от Simics
 - Ключевая фича – мощная подсистема памяти
 - Использовался для оценки многопроцессорных аппаратных систем (базы данные, веб)



Так что такое gem5?

- Michigan m5 + Wisconsin GEMS = gem5

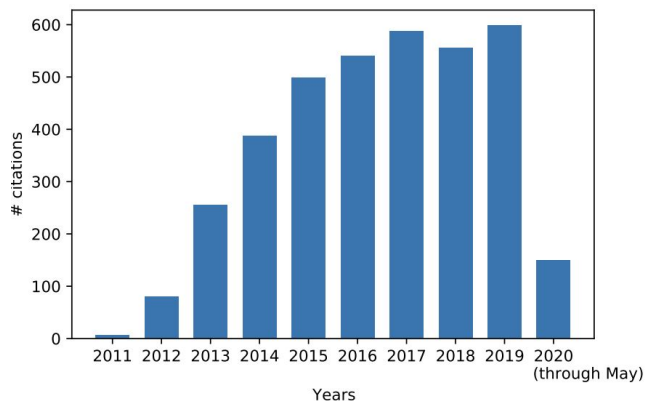
"The gem5 simulator is a modular platform for computer-system architecture research, encompassing system-level architecture as well as processor microarchitecture."

- Впервые представлен в SIGARCH, 2011
- На момент выхода поддерживал 6 (ARM, ALPHA, MIPS, Power, SPARC и x86) целевых архитектур
- На трех из них была поддержана загрузка Linux (ARM, ALPHA и x86)

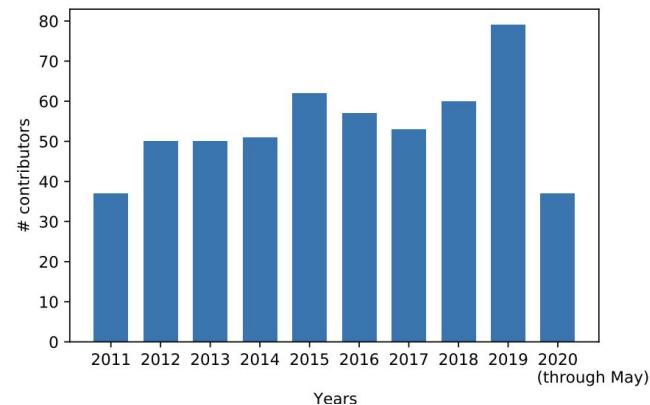
Мотивация



- Почему gem5?
 - Активное сообщество
 - Мощная симуляционная платформа



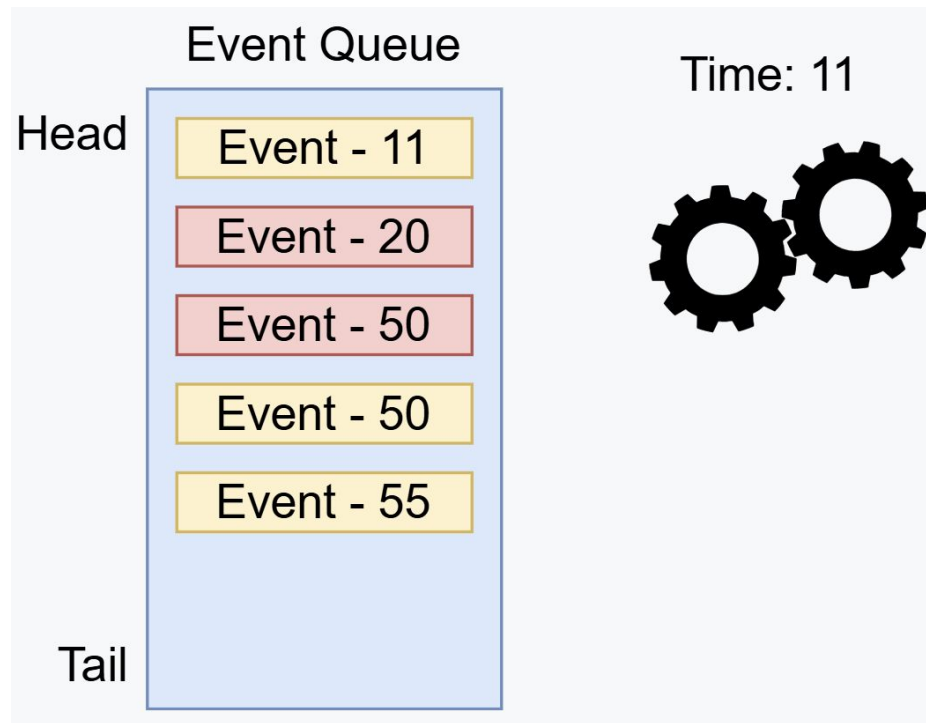
Количество цитирований оригинальной статьи (до 2020)



Количество контрибьюторов в проект (до 2020)

Тип симуляции

- gem5 основан на модели дискретных событий (DES)
- На каждом шаге симуляции (англ. timestep) gem5:
 - Событие в начале очереди извлекается
 - Событие выполняется
 - Новые события планируются
- Подробно про DES рассказывали в лекции про FSS или [здесь](#)



Визуализация очереди событий



Тип симуляции

- Timestep и time требует единицы измерения.
- В gem5 используется единица измерения, называемую «тиком» (англ. tick)
- Можно преобразовать «тик» симуляции в понятное пользователю время.
- Глобальная частота тиков симуляции обычно 1 пс/тик (или 10^{12} тиков/сек)

```
warn: The se.py script is deprecated. It will be removed in future releases of gem5.  
warn: Base 10 memory/cache size 8GB will be cast to base 2 size 8GiB.  
Global frequency set at 1000000000000 ticks per second  
src/arch/riscv/isa.cc:319: info: RVV enabled, VLEN = 256 bits, ELEN = 64 bits  
src/arch/riscv/linux/se_workload.cc:73: warn: Unknown operating system; assuming Linux.  
src/base/statistics.hh:279: warn: One of the stats is a legacy stat. Legacy stat is a stat that does not belong to any statistics::Group. Legacy  
stat is deprecated.  
system.remote_gdb: Listening for connections on port 7000  
**** REAL SIMULATION ****  
Exiting @ tick 885000 because exiting with last active thread context
```

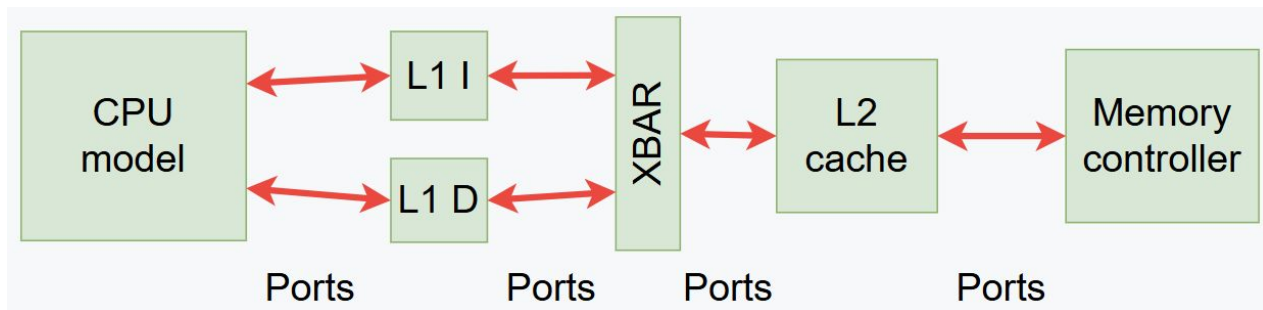
Частота симуляции

Итоговое время
симуляции

Стандартный лог запуска gem5 симуляции

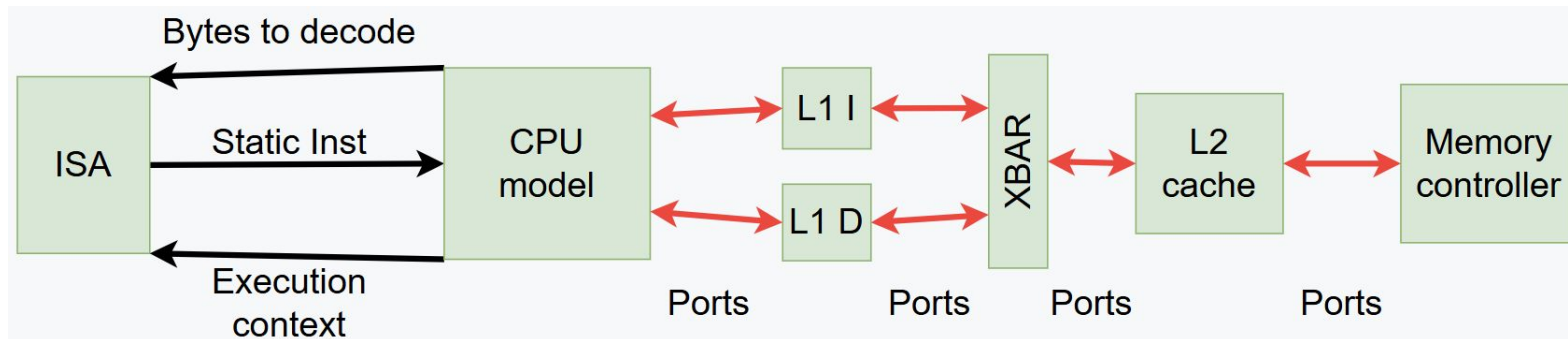
Архитектура памяти

- Порты (англ. ports) – главные интерфейсы памяти
- Порты позволяют отправлять запросы и получать ответы
- Порты являются однонаправленными (два типа: запрос/ответ)
- Любой объект с портом запроса может быть подключен к любому порту ответа
- Три режима доступа: timing, atomic, functional

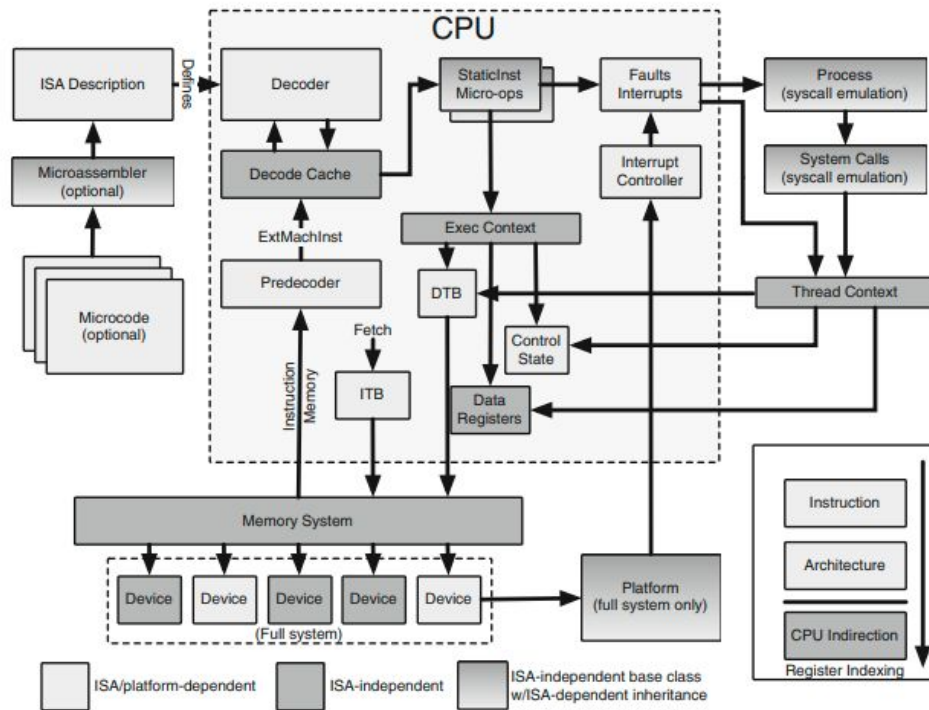


Архитектура CPU

- Архитектуры набора команд (ISA) и модели процессоров ортогональны.
- Любая архитектура набора команд должна работать с любой моделью процессора.
- Контекст выполнения (англ. Execution Context) — это интерфейс.



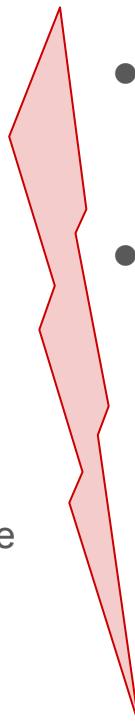
Архитектура детальнее



Детальная архитектура CPU&ISA

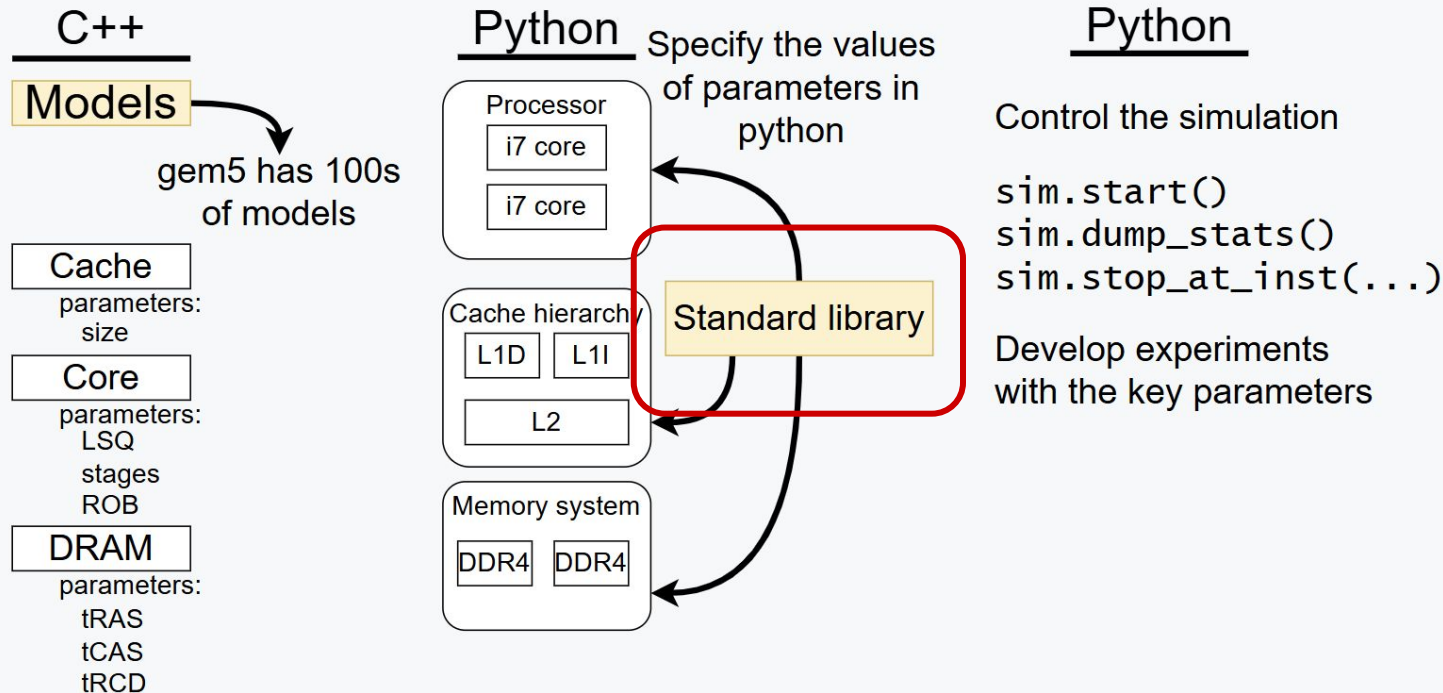
Ещё немного про дизайн CPU

- Декодер работает с объектами типа **StaticInst**
- Объект **StaticInst** содержит следующую информацию:
 - Класс операции
 - Регистрах источники и назначения
 - Флаги, указывающих на наличие микроопераций в инструкции
 - Функции, определяющих поведение инструкции



- Далее на основе **StaticInst** создается объект, характерный для **определенной** инструкции
- Объект **DynamicInst** содержит следующую информацию:
 - PC и предсказанный следующий PC
 - Результат исполнения
 - Номер потока (thread или же execution thread)
 - Номер ядра (core или же CPU)
 - Переименованные индексы регистров
 - Контекст исполнения (ExecContext)

Архитектура gem5



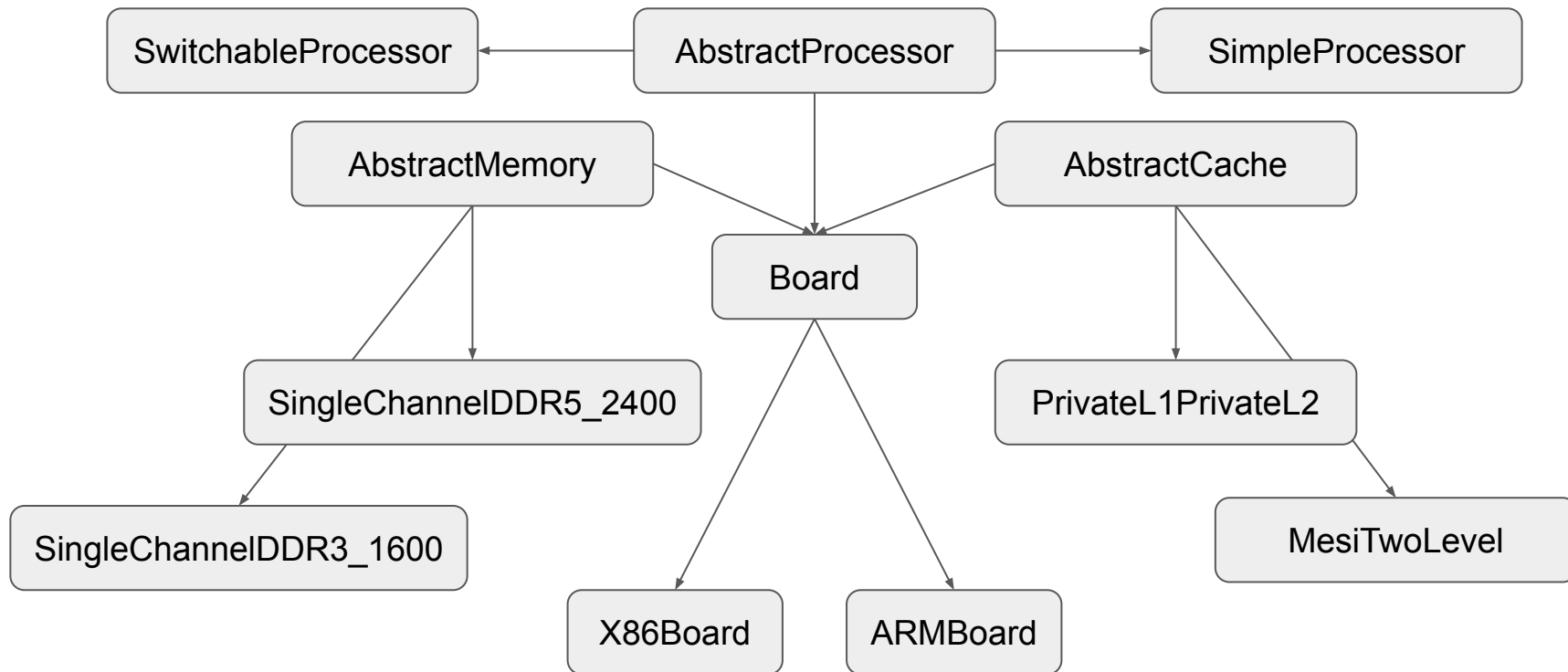
Стандартная библиотека



Уходим в глубину

- gem5 – модульный симулятор, который можно рассматривать как набор компонентов
- Чтобы уменьшить рутинную работу с gem5 базовыми компонентами (обычно называются **SimObject**), была введена стандартная библиотека
- Стандартная библиотека состоит из готовых скриптов Python, создающих экземпляры моделей, предоставляемых gem5
- Модели gem5 представляют собой мелкозернистые (англ. fine grained) концепции , которые не имеет смысла разделять дальше на подмодели

Компоненты





Компоненты

- Модели ядра gem5 располагаются в `src/cpu`
- Определяем базовые модели на C++:

```
class O3CPU : public BaseCPU  
{
```

- Стандартная библиотека gem5 располагается в `src/python/gem5`
- Компоненты конкретной симуляции определяем в Python-скрипте:

```
class i7CPU(O3CPU):  
    issue_width = 10
```



От слов – к делу

- Подключаем модули стандартной библиотеки:

```
from gem5.components.boards.simple_board import SimpleBoard
from gem5.components.processors.simple_processor import SimpleProcessor
from gem5.components.cachehierarchies.ruby.mesi_two_level_cache_hierarchy import
(
    MESITwoLevelCacheHierarchy,
)
from gem5.components.memory.single_channel import SingleChannelDDR4_2400
from gem5.components.processors.cpu_types import CPUTypes
from gem5.isas import ISA
from gem5.resources.resource import obtain_resource
from gem5.simulate.simulator import Simulator
```

- Полная реализация [здесь](#)

От слов — к делу

- Система кэшей

```
cache_hierarchy = MESITwoLevelCacheHierarchy(  
    l1d_size="16kB",  
    l1d_assoc=8,  
    l1i_size="16kB",  
    l1i_assoc=8,  
    l2_size="256kB",  
    l2_assoc=16,  
    num_l2_banks=1,  
)
```




От слов – к делу

- Система памяти:

```
memory = SingleChannelDDR4_2400()
```

Одноканальная система памяти на базе
модулей DIMM DDR4_2400_8x8

- По умолчанию для одного канала DDR4 установлено 8 Gb

От слов – к делу

- Ядро процессора:

```
processor = SimpleProcessor(  
    cpu_type=CPUtypes.TIMING,  
    isa=ISA.ARM,  
    num_cores=1)
```

Простая “in-order” модель:

- Single-cycle
- Инструкции, работающие с памятью берут задержку подсистемы памяти

От слов – к делу

- Плата:

```
board = SimpleBoard(  
    clk_freq="3GHz",  
    processor=processor,  
    memory=memory,  
    cache_hierarchy=cache_hierarchy,  
)
```

Плата, способная запускать любую ISA в режиме эмуляции системных вызовов (SE).



От слов – к делу

- Рабочая нагрузка:

```
board.set_workload(observe_resource("arm-gapbs-bfs-run"))
```

- Создаём модель:

```
simulator = Simulator(board=board)  
simulator.run()
```

- Запуск:

```
> gem5-mesi 01-components.py
```

Стандартная библиотека

- gem5 stdlib располагается в `src/python/gem5`
- Различают два типа объектов:
 - Components – модули для построения системы
 - Prebuilt – полноценные системы с заданными параметрами

```
src/python/gem5 on  stable [!?] via  v3.10.12  
> tree -d -L 2
```

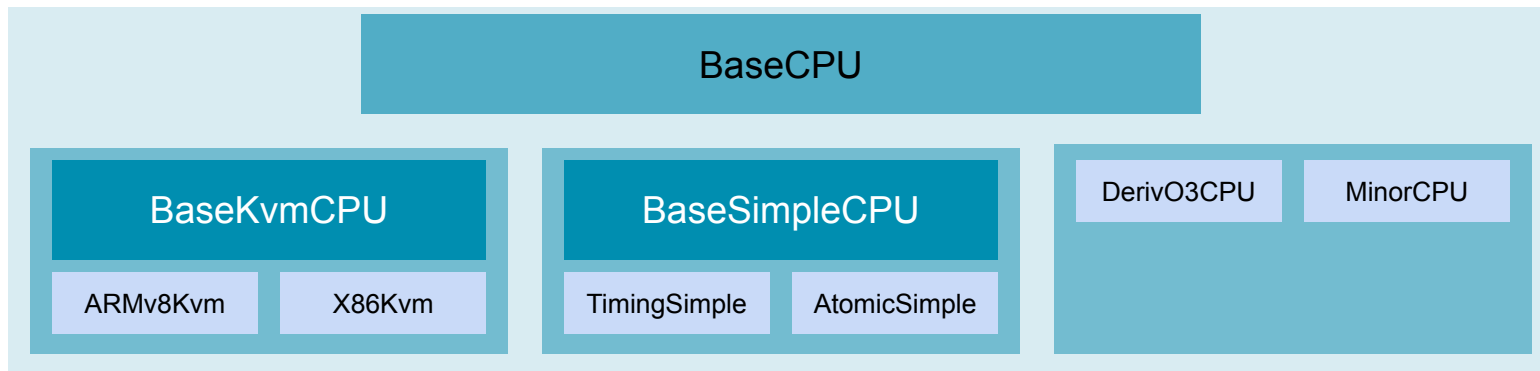
```
.  
├── components  
│   ├── boards  
│   ├── cachehierarchies  
│   ├── devices  
│   ├── memory  
│   └── processors  
├── prebuilt  
│   ├── demo  
│   ├── riscvmatched  
│   └── viper  
├── resources  
│   └── client_api  
├── simulate  
├── utils  
│   ├── multiprocessing  
│   └── multisim
```

Модель ЦП

CPU Models



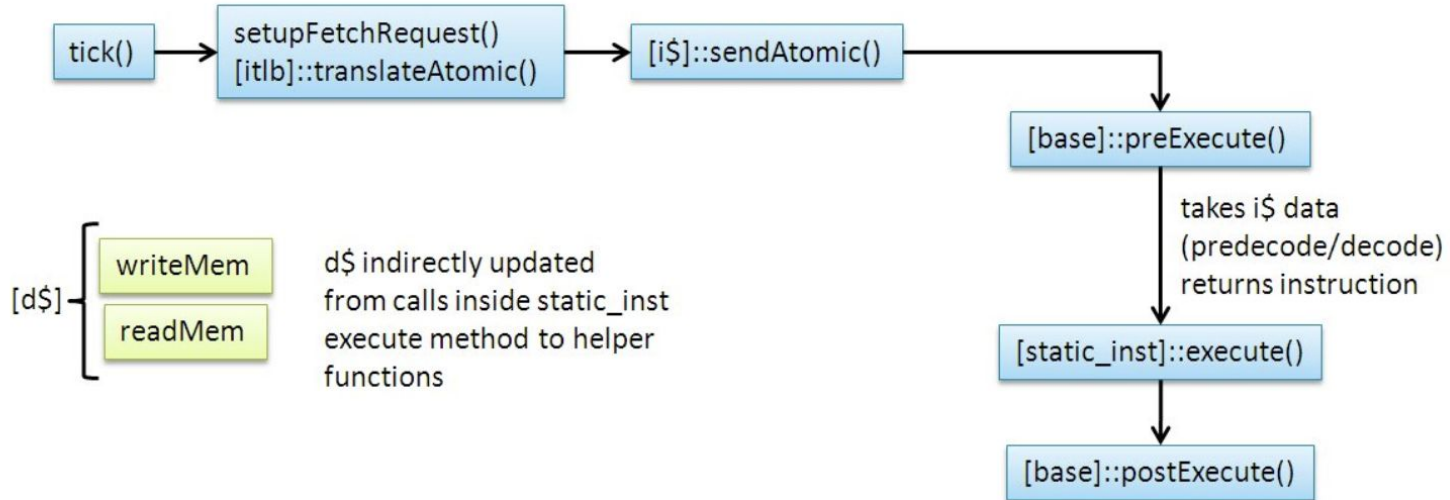
- В gem5 есть разные модели процессора в зависимости от детализации:



Доступные модели ЦП в симуляторе gem5

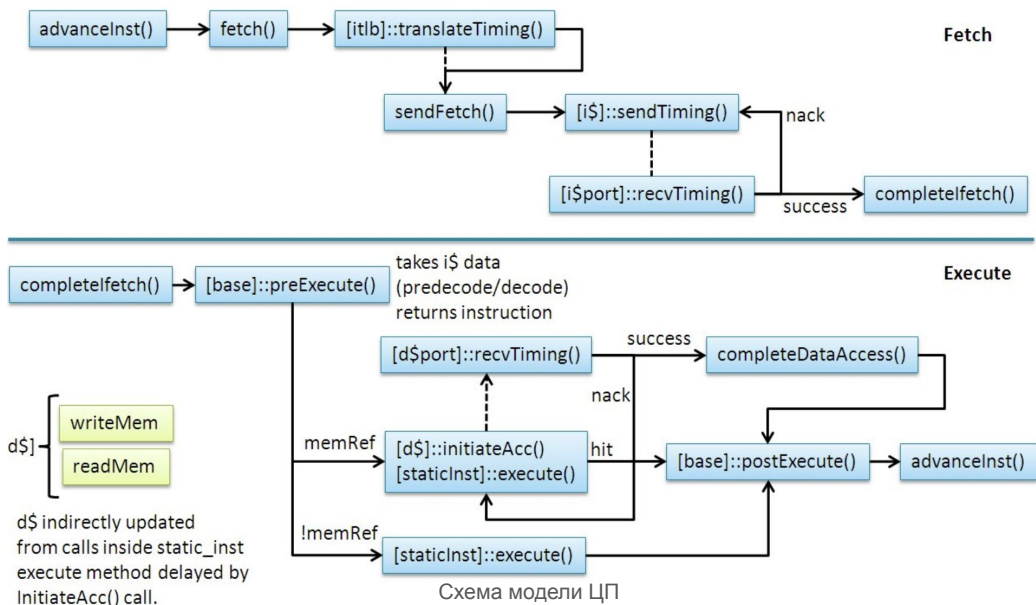
SimpleCPU

- AtomicSimpleCPU:
 - Последовательность вложенных вызовов
 - Использование: прогрев, fast-forwarding



SimpleCPU

- TimingSimpleCPU:
 - Разделение транзакций
 - Модели задержки в очереди и конкуренция за ресурсы



O3CPU

- Синхронизация доступа к памяти с семантикой выполнения внутри выполнения
- Временные буферы между этапами

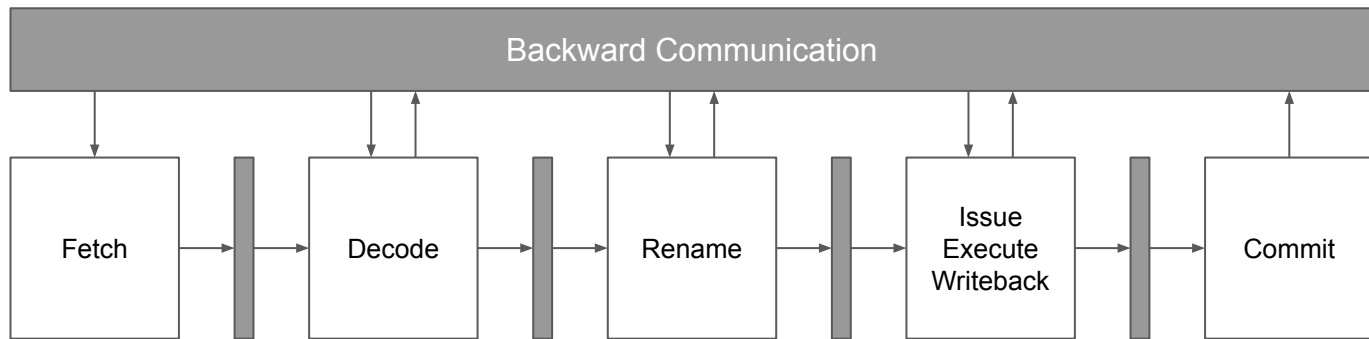


Схема внеочередного исполнения в симуляторе gem5

KvmCPU

- KVM – Kernel-based virtual machine
- Используется для выполнения нативных операций на хозяйских платформах x86 и ARM
- Гостевая и хозяйская системы должны иметь одинаковую архитектуру набора команд
- Очень полезна для функционального тестирования и ускоренной перемотки.

Summary

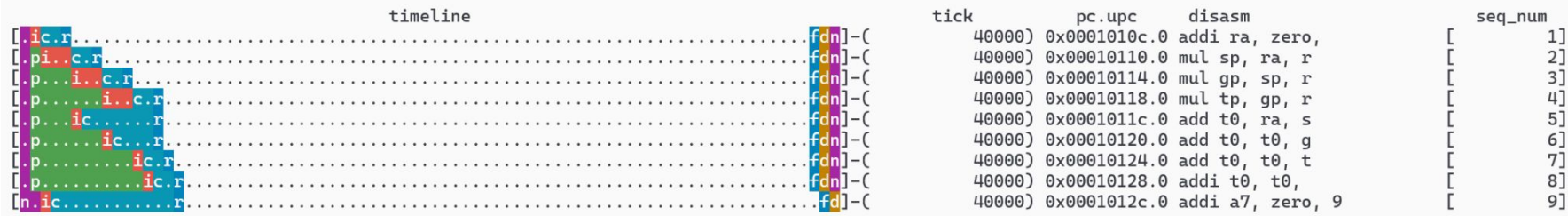
Модель	Скорость	Задержки	Детальность
BaseKvmCPU	Очень быстрый	-	Нет кэшей, нет BP
BaseSimpleCPU	Быстрый	Простая модель задержек	Кэши, ограниченный BP
DerivO3CPU	Медленный	Детальные задержки	Детальные кэши, BP

Визуализация

- [PipeView](#)

- + Простая визуализация в терминальной псевдографике
- + Интегрировано в gem5
- Масштабирование отсутствует
- Ограниченное интерактивное исследование
- Некорректное отображение на временной шкале

// f = fetch, d = decode, n = rename, p = dispatch, i = issue, c = complete, r = retire



Визуализация простой программы с
помощью O3PipeView

Визуализация

- [Konata](#):
 - + Графический интерфейс
 - + Интерактивное исследование и масштабирование
 - Рассинхронизация с gem5
 - Нет фильтров по стадиям/событиям конвейера

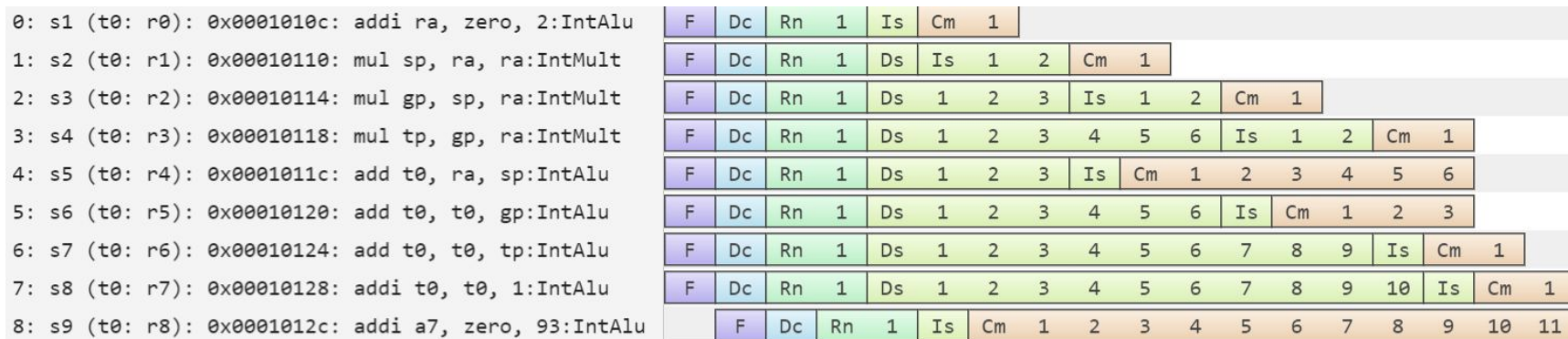


Рис. 4. Визуализация простой программы
с помощью [Konata](#)

Домашнее задание №2

1. Выбрать три политики кэширования из четырех (Random, LFU, LRU, MRU), провести замеры на [riscv-benchmarks](#) с O3 –cpu-type, собрать статистику:
 - a. IPC
 - b. Cache miss rate
 - c. Средняя задержка промаха L1D кэша**1*** Варьировать L1 size, L2 size для лучшей политики

2. Выбрать три политики предсказания из четырех (LocalBP, BiMode, TAGE SC L8 kb, TournamentBP), провести замеры на [riscv-benchmarks](#) с O3 –cpu-type, собрать статистику:
 - a. IPC
 - b. MPKI (mispredictions per kilo instructions)
 - c. Количество сброшенных инструкций на 1000 инструкций**2*** Варьировать размер истории для лучшей политики

3. Реализовать политику X для Y. Провести аналогичные замеры.

Формат сдачи – PDF-документ с краткими пояснениями полученных результатов и графиками.

Ссылки

1. [M5 Simulator](#)
2. [GEMS](#)
3. [Актуальная статья по gem5](#)
4. [Оригинальная статья по gem5](#)
5. [gem5 Bootcamp website – Lectures & Tutorials](#)